

Consolidado Canônico do Projeto Olympus Climb (Base Vigente)

0. Escopo, precedência e validade documental

Este documento passa a ser a fonte de verdade operacional do Olympus Climb até nova revisão formal.

Ordem de precedência aplicada:

1. `OLYMPUS CLIMB 2026` (base principal canônica).
2. `GitFlow — Olympus Climb` (somente regras ainda válidas após adaptação).
3. `GAMEPLAN OLYMPUS CLIMB - 2025` (somente legado técnico reaproveitável).
4. Conclusões da auditoria consolidada (resolução de conflitos/lacunas).

Classificação oficial:

- `OLYMPUS CLIMB 2026`: canônico (complementado aqui para executabilidade).
- `GitFlow`: complementar (parcialmente adaptado; versão antiga não é soberana).
- `GAMEPLAN 2025`: legado útil (superseded como plano de execução).

Decisões de descarte explícito:

- Está descartada a execução por “fases cronológicas” como eixo de engenharia.
- Está descartado branch naming por `feature/fase-X`.
- Está descartada identidade scraper-first.
- Está descartado tratar mobile/dashboard/scraper como produtos isolados.

1. Visão do produto

O Olympus Climb é um sistema agentic de operação de carreira com controle humano, auditabilidade e aprendizado contínuo, projetado para:

- operar o funil real de descoberta, análise, decisão e execução de candidaturas;
- funcionar como laboratório disciplinado de engenharia agentic em produção pessoal.

O produto não é:

- scraper isolado;
- app mobile isolado;
- dashboard isolado;
- coleção de automações ad hoc.

2. Princípio arquitetural central

Regra central vigente:

- o valor do sistema está na qualidade da decisão e da execução auditável, não na coleta de vagas por si só.

Todo módulo novo deve justificar impacto em pelo menos um eixo:

- qualidade de decisão;
- qualidade de execução;
- redução de retrabalho;
- preservação de contexto;
- confiabilidade agentic;
- capacidade de aprendizado histórico.

3. Capacidades-mãe (organização oficial)

O backlog, branches e specs passam a ser organizados por capacidade:

1. Descoberta.
2. Normalização.
3. Enriquecimento.
4. Match.
5. Estratégia.
6. Execução.
7. Memória.
8. Observabilidade.

4. Camadas do sistema (arquitetura lógica oficial)

1. Core Data Layer: estado canônico persistente.
2. Agent Runtime Layer: orquestração agentic, handoff, sessão, guardrails.
3. Skills/Tools Layer: capacidades invocáveis e integráveis.
4. Human Oversight Layer: aprovação/revisão de ações sensíveis.
5. Operator Interfaces Layer: mobile cockpit e interfaces operacionais.

Regra de governança:

- nenhuma feature cruza camadas sem contrato explícito.
- acoplamento transversal sem spec é proibido.

5. Core de dados (estado canônico)

Entidades centrais obrigatórias (v1):

- `JobSource`
- `Company`
- `JobPosting`
- `ResumeProfile`
- `CoverAsset`

- `Application`
- `AgentRun`
- `DecisionLog`
- `MemoryEntry`
- `SkillExecution`

Regras obrigatórias:

- todo evento decisório relevante gera `AgentRun` e/ou `DecisionLog`.
- toda execução de capacidade gera rastro em `SkillExecution`.
- deduplicação de vaga é mandatória por hash e chave de origem.
- nenhum agente escreve estado crítico sem trilha auditável.

6. Runtime agentic (política oficial)

Princípios:

- proibido agente “deus”.
- agentes especialistas com responsabilidade estreita.
- handoff explícito e rastreável.
- retries com limite, fallback e evidência.

Contratos mínimos de runtime:

- input versionado;
- output validado;
- contexto de execução identificável;
- custo/tempo por execução registrado;
- decisão explicável quando houver decisão automática.

7. Papel dos agentes (vigente)

Agentes base oficiais:

1. `Scout Agent`: descoberta e priorização de coleta.
2. `Normalizer Agent`: transformação para schema canônico.
3. `Enrichment Agent`: contexto institucional/técnico/risco.
4. `Match Agent`: aderência técnica e estratégica explicada.
5. `Strategy Agent`: decisão operacional proposta.
6. `Execution Agent`: execução concreta com evidência.
7. `Memory Agent`: consolidação/recuperação contextual.
8. `Review Agent`: revisão de alto impacto e bloqueio preventivo.

Regra:

- agente não invade responsabilidade de outro sem ADR de exceção.

8. Papel das skills (vigente)

Skill é unidade formal de capacidade operacional.

Toda skill deve ter:

- propósito único;
- contrato de entrada;
- contrato de saída;
- regras e limites;
- logs estruturados;
- testes;
- evals;
- exemplos reais;
- fallback.

Estrutura mínima mandatória:

- `README.md`
- `input.schema.ts`
- `output.schema.ts`
- `handler.ts`
- `evals/`
- `fixtures/`
- `tests/`
- `examples/`

9. MCP: uso e critério de adoção

MCP é padrão preferencial de integração quando evita acoplamento interno.

Adoção permitida somente se:

- reduz acoplamento estrutural;
- padroniza integração que viraria exceção frágil;
- aumenta reuso;
- mantém operabilidade simples.

Adoção bloqueada se:

- adiciona complexidade sem ganho mensurável;
- cria lock-in operacional desnecessário;
- não oferece fallback claro.

Toda nova integração MCP exige:

- ADR de adoção;
- spec de contrato;
- task manifest de rollout;
- plano de fallback/desligamento.

10. Política de human approval

Ações com aprovação humana obrigatória:

- envio de candidatura;
- envio de e-mail;
- mensagem para recrutador;
- alteração destrutiva de dados;
- abordagem inédita de alto risco;
- resposta aberta de alto impacto.

Ações autônomas permitidas:

- descoberta;
- parsing;
- normalização;
- enriquecimento;
- scoring;
- deduplicação;
- ranking;
- agendamento interno;
- atualização de memória/logs.

Regra:

- se risco operacional subir e não houver regra explícita, default é exigir aprovação.

11. Memória do sistema

Memória deve ser útil, seletiva e auditável.

Guardar:

- empresas analisadas;
- vagas semelhantes;
- decisões e rationale;
- respostas eficazes;
- padrões de rejeição/alinhamento;
- versões de currículo e resultado.

Não guardar sem critério:

- lixo transitório;
- redundância sem valor;
- inferência frágil tratada como fato.

12. Observabilidade e auditoria

Toda ação relevante deve responder:

- quem executou;
- quando;
- com qual contexto;
- com quais tools;
- qual decisão;
- qual saída;
- qual custo;
- qual evidência.

Logs mínimos obrigatórios:

- agente;
- skill;
- fila;
- aprovação;
- execução externa;
- erro;
- decisão.

Métricas mínimas obrigatórias:

- vagas por fonte;
- taxa de deduplicação;
- taxa de vagas aproveitáveis;
- taxa de aplicações enviadas;
- taxa de aprovação humana;
- taxa de execução automática bem-sucedida;
- tempo médio por decisão;
- tempo médio por candidatura;
- custo por ciclo agentic;
- conversão por variante de currículo.

13. Evals e confiabilidade

Regra canônica:

- sem eval, não há promoção de capacidade para uso operacional confiável.

Eixos mínimos de avaliação:

- extração;
- normalização;
- match;
- rationale;
- segurança operacional;
- robustez de execução;
- utilidade de sugestões;
- consistência histórica.

Todo agente/skill crítico deve ter:

- casos bons;
- casos ruins;
- casos ambíguos;
- critérios explícitos de sucesso/falha.

14. Automação de navegador

Automação de navegador é capacidade de execução, não identidade do sistema.

Regras:

- priorizar caminhos estáveis;
- toda automação com logs e evidência;
- etapa crítica de formulário sob aprovação humana;
- fallback obrigatório para DOM instável.

15. Organização do repositório (alvo oficial)

Estrutura oficial de referência:

- `apps/`
- `services/`
- `packages/`
- `infrastructure/`
- `docs/`

Diretriz:

- `doc/` (PDFs legados) permanece como arquivo histórico.
- `docs/` passa a ser fonte viva e canônica.

16. Organização oficial de skills

Local padrão:

- `packages/skills/<skill-name>/`

Cada skill deve ser isolável, testável e reutilizável sem dependência implícita de contexto externo.

17. Stack oficialmente mantida

Stack mantida oficialmente (reaproveitada do 2025 e reafirmada no 2026):

- TypeScript
- Node.js + NestJS + Fastify
- PostgreSQL + Prisma
- Redis + BullMQ
- React Native + Expo + NativeWind
- PM2
- Jest
- Artillery
- TypeDoc (com condições de adoção, seção 24)

Origem explícita:

- manutenção técnica herdada do `GAMEPLAN 2025`;
- reposicionamento arquitetural definido por `OLYMPUS 2026`.

18. Diretrizes de engenharia (vigentes)

1. Desenvolvimento orientado por spec.
2. Tarefas pequenas, rastreáveis e validadas.
3. Contratos explícitos antes de integração.
4. Menor entrega útil primeiro.
5. Evitar overengineering por antecipação.
6. Evitar under-specification em fronteiras críticas.
7. Toda decisão relevante com rastro documental.

19. Política de Git e branches (atualizada)

Regras mantidas do GitFlow (válidas):

- `main` protegida;
- zero push direto na `main`;
- PR obrigatório;
- CI como gate;
- revisão obrigatória;
- versionamento SemVer.

Regras substituídas:

- branch por fase é revogada.
- branch por capacidade/eixo é obrigatória.

Padrões aceitos:

- `feature/capability-<nome>`
- `feature/agent-<nome>`
- `feature/skill-<nome>`
- `feature/integration-<nome>`
- `feature/reliability-<nome>`
- `feature/ui-<nome>`

- `feature/eval-<nome>`
- `bugfix/<nome>`
- `hotfix/<nome>`

`develop`:

- opcional e proibida por padrão.
- só pode ser adotada com ADR explícita justificando benefício real.

20. ADRs obrigatórias (gate documental)

Antes de implementação funcional pesada, devem existir ADRs aceitas para:

1. Modelo arquitetural agent-centric.
2. Estratégia de branch e governança de PR.
3. Modelo de dados canônico v1.
4. Política de human approval.
5. Contrato formal de skills.
6. Política de observabilidade/auditoria.
7. Política de adoção MCP.
8. Política de evals e promoção de capacidades.

21. Specs obrigatórias (gate de execução)

Specs mínimas obrigatórias:

1. Core Data v1.
2. Ingestion v1 (search/open/extract/normalize/deduplicate).
3. Match + Strategy v1.
4. Approval Queue v1.
5. Execution Evidence v1.
6. Observability v1.
7. Evals v1.
8. Branch/PR Governance v1 (operacional).
9. Memory Lifecycle v1.

22. Task manifests obrigatórias (gate de trabalho)

Toda spec executável deve quebrar em manifests com:

- objetivo único;
- inputs necessários;
- alvos;
- dependências;
- restrições;
- Definition of Done;
- validação;

- evidência;
- risco.

Regra:

- task sem manifest não entra em execução de engenharia pesada.

23. Prontidão para iniciar implementação

Classificações oficiais:

- `não pronto`
- `parcialmente pronto`
- `pronto com ressalvas`
- `pronto`

Critério para sair de `não pronto`:

- matriz de canonicidade oficial publicada;
- ADRs mínimas aceitas;
- specs mínimas publicadas;
- task manifests iniciais prontas;
- política Git atualizada em documento vigente.

Estado atual consolidado:

- `não pronto para implementação funcional pesada`;
- `pronto para ciclo de formalização executável` (ADRs/specs/manifests).

24. Papel do TypeDoc

TypeDoc é obrigatório como documentação viva após:

- boundaries de módulos definidas;
- contratos públicos estabilizados;
- convenção TSDoc formalizada.

TypeDoc é adiado enquanto arquitetura de pacotes/contratos estiver em churn alto.

25. Declaração final do sistema

O Olympus Climb é oficialmente um sistema agentic, capability-driven, spec-driven, com core de dados canônico, agentes especialistas, skills formais, integrações controladas via tools/MCP quando justificadas, aprovação humana em ações sensíveis, observabilidade auditável e evolução incremental orientada por evidência.

Qualquer artefato anterior que conflite com este consolidado perde vigência operacional.